PC

program counter

holds an address

fold

You are the PC, the program counter

You hold: the address of the next instruction to be fetched.

1. Start the program with **0** on your board.
2. When the CU says "Fetch", read your address out loud.
3. As soon as the MDR has written down the instruction, add 1 and write your new address.

Check it. After every fetch, your number is exactly 1 more than the MAR's.

MAR

memory address register

holds an address

fold

You are the MAR, the memory address register

You hold: the address of the memory location being read from or written to.

1. When the PC reads out its address, write it on your board and call it to memory: "Address 0!"
2. When the CU gives you an address, write it and call it out in the same way.

Your rule. You are the one who calls an address to memory.

Check it. Your number is always an address: a whole number from 0 to 7.

MDR

memory data register

holds data

fold

You are the MDR, the memory data register

You hold: the data or instruction just fetched from memory, or about to be written to it.

1. When a memory location reads out its contents, write them on your board.
2. In decode, read the instruction out to the CU.
3. For STORE: copy the accumulator's number onto your board, walk it to the address the MAR called, and tell that location the number.

Your rule. You are the one who carries things between the CPU and memory.

Check it. What is on your board came from, or is going to, the address on the MAR's board.

CU

control unit

controls

fold

You are the CU, the control unit

You hold: no value. You give the orders.

1. Say "Fetch" to start every cycle.
2. When the MDR reads you the instruction, decode it: say what it means. "LOAD 5: copy the number at address 5 into the accumulator."
3. Execute it with the orders below. Then say "Fetch" again.

LOAD *n* "MAR, address *n*." Once the MDR has the number: "Accumulator, copy it."

ADD *n* "MAR, address *n*." Once the MDR has the number: "ALU, add."

STORE *n* "MAR, address *n*. MDR, take the accumulator's number there."

STOP "Stop. The program has ended."

Check it. You give every order, and the ALU does every sum.

ALU

arithmetic logic unit

calculates and compares

fold

You are the ALU, the arithmetic logic unit

You hold: no value of your own. You do the calculations, and the comparisons and logic (AND, OR, NOT).
Today's programs use adding.

1. When the CU says "ALU, add", take the accumulator's number and the MDR's number.
2. Say the sum out loud: "12 plus 30 is 42."
3. Give the answer to the accumulator.

Check it. You do every sum, and the accumulator keeps the answer.

ACC

accumulator

holds data

fold

You are the ACC, the accumulator

You hold: the result of the calculation the ALU has just done.

1. Start the program with an empty board.
2. For LOAD: when the CU says "Accumulator, copy it", write the MDR's number.
3. For ADD: when the ALU gives you its answer, write it in place of your old number.
4. For STORE: show your number to the MDR.

Check it. Your number changes on LOAD and ADD, and stays the same for STORE and STOP.

main memory · address

0

fold

You are address 0 in main memory (RAM)

RUN 1

LOAD 5
instruction

RUN 2

LOAD 6
instruction

When the MAR calls "Address 0!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

1

fold

You are address 1 in main memory (RAM)

RUN 1

ADD 6

instruction

RUN 2

ADD 5

instruction

When the MAR calls "Address 1!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

2

fold

You are address 2 in main memory (RAM)

RUN 1

STORE 7
instruction

RUN 2

ADD 5
instruction

When the MAR calls "Address 2i", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

3

fold

You are address 3 in main memory (RAM)

RUN 1

STOP
instruction

RUN 2

STORE 7
instruction

When the MAR calls "Address 3!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

4

fold

You are address 4 in main memory (RAM)

RUN 1

—

empty

RUN 2

STOP

instruction

When the MAR calls "Address 4!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

5

fold

You are address 5 in main memory (RAM)

RUN 1

12

data · stored as 00001100

RUN 2

12

data · stored as 00001100

When the MAR calls "Address 5!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

6

fold

You are address 6 in main memory (RAM)

RUN 1

30

data · stored as 00011110

RUN 2

30

data · stored as 00011110

When the MAR calls "Address 6!", read out your contents for this run. Your card stays with you: reading memory copies what is stored, so it is still here afterwards.

Check it. You speak when your own address is called.

main memory · address

7

fold

You are address 7 in main memory (RAM)

RUN 1

empty to start · write the number here

RUN 2

empty to start · write the number here

For STORE 7, the MDR brings you a number. Write it in the box for this run, in pencil. If the MAR calls "Address 7!" after that, read it out.

Check it. At STOP, your box holds the number the program worked out.